



PYVERSE ARCADE

Complete Program Documentation

How to Build It Yourself — Step by Step

▣ [Rollercoaster](#) | ▣ [Treasure Island](#) | ▣ [Space Mission Control](#)

Institute: Benazir Bhutto Shaheed Human Resource Research & Development Board

Course: Python Programming with AI

Program Overview

PyVerse Arcade is a menu-driven Python program that contains three fully independent games inside one file. When the program starts, the user sees a main menu and picks which game they want to play. After finishing, they can go back to the menu or exit.

Every game is written inside a def function. The main menu loop at the bottom calls whichever function the player picks. This keeps the code organized and easy to read.

Program Name	PyVerse Arcade
Language	Python 3
Total Games	3
Concepts Used	def functions, while loop, for loop, if/else, nested if/else, slicing, input()
Structure	Main menu loop → calls game functions → returns to menu
File	pyverse_arcade.py

Where Each Concept Is Used

Before building anything, understand which concept appears in which game. This table is your map.

Concept	❑ Rollercoaster	❑ Treasure Island	❑ Space Mission
def function	✓ rollercoaster()	✓ treasure()	✓ space()
while loop	✓ height, age, photo validation	✓ play again loop	✓ name, health, fuel validation
for loop	✓ session summary	✗ not used	✓ equipment checklist
if / else	✓ height check, age groups	✓ left/right, door choice	✓ health check, destination
Nested if / else	✓ senior inside age check	✓ swim/wait inside left	✓ launch verdict (3 layers)
Slicing	✗ not used	✗ not used	✓ name[:3].upper() → call sign
input()	✓ height, age, photo, again	✓ direction, action, door, again	✓ name, health, dest, fuel, items

Step 1 — Build the Main Menu

Before writing any game, build the main menu first. This is the shell that holds everything together. The menu is a while True loop that keeps showing until the player picks Exit.

How the Main Menu Works

1. Print the arcade menu with 4 choices.
2. Ask the user to type 1, 2, 3, or 4.
3. A while loop repeats the question if they type something invalid.
4. If they pick 4, print Goodbye and break out of the loop.
5. Otherwise call the right game function.
6. When the game finishes, ask if they want to go back to the menu.

Flowchart

```

Start
|
v
Show main menu (1. Rollercoaster  2. Treasure Island  3. Space Mission  4.
Exit)
|
v
While input is not 1/2/3/4  ->  keep asking
|
v
Pick 4?
|-- Yes --> Print Goodbye  -->  END
|
v
Pick 1  ->  call rollercoaster()
Pick 2  ->  call treasure()
Pick 3  ->  call space()
|
v
Game finishes
|
v
Back to main menu? (yes/no)
|-- yes  -->  loop back to top
|-- no   -->  Print Goodbye  -->  END

```

Code

```

while True:
    print("┌───────────────────────────────────────────────────────────────────────────────────┐")
    print("│                □ PYVERSE ARCADE □                │")
    print("└───────────────────────────────────────────────────────────────────────────────────┘")
    print("│  1.  □ Rollercoaster                                │")
    print("│  2.  □ Treasure Island                              │")
    print("│  3.  □ Space Mission Control                        │")
    print("│  4.  □ Exit                                          │")
    print("└───────────────────────────────────────────────────────────────────────────────────┘")
    print()

    while True:
        pick = input("Choose a game (1/2/3/4): ")
        if pick in ["1", "2", "3", "4"]:
            break
        print("Please enter 1, 2, 3, or 4.")

```

```

if pick == "4":
    print("Goodbye!")
    break

print()

if pick == "1":
    rollercoaster()
elif pick == "2":
    treasure()
elif pick == "3":
    space()

print()
while True:
    action = input("Back to main menu? (yes/no) ").lower()
    if action in ["yes", "no"]:
        break
    print("Please type yes or no.")

if action == "no":
    print("Goodbye!")
    break

```

Requirements

- **while True** outer loop — keeps the menu running after each game
- **while True** inner loop — validates the menu choice (1/2/3/4 only)
- **if/elif/else** — decides which game function to call
- **break** — exits the outer loop when player picks Exit or says no to menu

GAME 1 — ROLLERCOASTER BOOKING SYSTEM

Multi-customer loop | Age groups | Session summary

What This Game Does

The Rollercoaster game is a booking system that can process multiple customers one after another. For each customer it checks their height, calculates a ticket price based on age (with a special senior discount), asks about photos, prints a bill, and then asks if there is another customer. At the very end it prints a full session summary using a for loop.

Concepts Used in This Game

- **while True** — outer customer loop, repeats for every new customer
- **while True** — three inner validation loops (height, age, photo)
- **if / elif / else** — height gate and age group pricing
- **Nested if/else** — senior check (45-55) sits INSIDE the adult elif block
- **for loop** — loops over the summary data list to print each stat
- **Variables** — count, revenue, seniors, photos track totals across all customers

Flowchart

```
def rollercoaster():  
    |  
    v  
    Set count=0, revenue=0, seniors=0, photos=0  
    |  
    v  
    OUTER while True (one loop per customer)  
    |  
    v  
    count += 1 --> print Customer {count}  
    |  
    v  
    While height input invalid -> keep asking  
    |  
    v  
    Height > 120cm?  
    |-- No --> Sorry, too short  
    |  
    v  
    Yes --> You can ride!  
    |  
    v  
    While age input invalid -> keep asking  
    |  
    v
```

```

Age?
|-- Under 12          --> price = $5    (Child)
|-- 12 to 18          --> price = $7    (Teenager)
|-- 45 to 55          --> price = $0    (Senior FREE) ★ seniors += 1
|-- 18 or over        --> price = $12   (Adult)
|
v
While photo input invalid -> keep asking
|
v
Want photos?
|-- yes --> photo = $3    photos += 1
|-- no  --> photo = $0
|
v
total = price + photo
revenue += total
Print bill
|
v
Process another customer? (yes/no)
|-- yes --> loop back to top
|-- no  --> break
|
v
for label, value in data:  --> print each summary line
|
v
Print average revenue
|
END

```

Key Things to Remember

- **Senior check order matters:** write `elif 45 <= a <= 55` BEFORE `elif a > 18`, otherwise seniors get charged \$12
 - **Counting variables:** `count`, `revenue`, `seniors`, `photos` must be set to 0 BEFORE the while loop starts
 - **revenue += total** — this runs inside the `height > 120` block only. Customers who can't ride don't add to revenue
 - **for loop summary:** store your summary rows in a list of tuples, then loop over them to print neatly
-

□ GAME 2 — TREASURE ISLAND

Decision tree adventure | Nested if/else | Play again loop

What This Game Does

Treasure Island is a short text adventure where every answer the player types leads to a different outcome. The whole game is built from nested if/else blocks — one decision lives inside another. If the player wins or loses they are asked if they want to play again.

Concepts Used in This Game

- **while True** — wraps the whole game so play again works
- **if / else** — first decision: left or right
- **Nested if / else level 2** — inside left: swim or wait
- **Nested if / else level 3** — inside wait: red, blue, or yellow door
- **input()** — four separate inputs: direction, action, door, play again

Flowchart

```
def treasure():
    |
    v
    OUTER while True    (play again loop)
    |
    v
    input: Go left or right?
    |
    |-- right -->  You fall into a hole. Game Over.
    |
    v
    left -->  Path opens to a river.
    |
    v
    input: Do you swim or wait?
    |
    |-- swim -->  Attacked by trout. Game Over.
    |
    v
    wait -->  Cave with three doors.
```

```

|
v
input: Which door? (red / blue / yellow)
|-- red      -->  Burned by fire. Game Over.
|-- blue     -->  Eaten by beasts. Game Over.
|-- yellow   -->  ★ You found the treasure! You win!
|-- other    -->  Invalid choice. Game Over.
|
v
Play again? (yes/no)
|-- yes      -->  loop back to top
|-- no       -->  break -->  END

```

Key Things to Remember

- **The whole game is nested:** the door choice is inside the wait block, which is inside the left block. If you indent wrong, the logic breaks
- **Only one winning path:** left → wait → yellow. Everything else ends the game
- **Play again uses the same choice variable:** be careful — the game uses choice for both direction and swim/wait. If confused, use different variable names like dir and action
- **break vs return:** since this code is inside a def function, using break exits the while loop, which then exits the function back to the main menu

❏ GAME 3 — SPACE MISSION CONTROL

Slicing | for loop checklist | 3-layer nested if/else launch verdict

What This Game Does

Space Mission Control is a multi-stage clearance game. The player registers as an astronaut (name is sliced into a call sign), passes a health screening, picks a destination, completes an equipment checklist using a for loop, enters a fuel level, and finally receives a launch verdict from a 3-layer nested if/else block.

Concepts Used in This Game

- **Slicing** `name[:3].upper()` — first 3 letters of name become the call sign
- **while True** — outer play again loop, plus 3 validation loops (name, health, fuel)

- **if / else** — health check gates the entire game (below 75 → rejected)
- **if / elif / else** — destination choice sets risk level and fuel requirement
- **for loop** — loops over equipment list, asks ready for each item
- **Nested if/else (3 layers)** — layer 1: equipment ready?, layer 2: fuel enough?, layer 3: risk level

Flowchart

```
def space():
    |
    v
OUTER while True (play again loop)
    |
    v
While name input invalid -> keep asking
    |
    v
sign = name[:3].upper()    <-- SLICING
    |
    v
While health input invalid -> keep asking
    |
    v
Health >= 75?
|-- No    --> Health check failed. You are not cleared. --> skip to play
again
    |
    v
Yes --> Health check passed!
    |
    v
input: Choose destination (moon / mars / jupiter)
    |
|-- moon    --> risk=LOW    need=50% emoji=☐
|-- mars    --> risk=MEDIUM need=75% emoji=☐
|-- jupiter --> risk=HIGH   need=90% emoji=☐
    |
    v
for item in [Oxygen Tank, Space Suit, Navigation Computer]:
    While input not yes/no -> keep asking
    yes --> print OK
    no  --> print MISSING --> ready = False
    |
    v
While fuel input invalid -> keep asking
```

```

|
v
Print launch report
|
v
NESTED IF/ELSE (layer 1)
Equipment ready?
|-- No --> (layer 2)
|
|       Fuel ok?
|
|       |-- Yes --> ABORTED: equipment incomplete
|       |-- No --> ABORTED: multiple issues
|
|-- Yes --> (layer 2)
|
|       Fuel >= need?
|       |-- No --> ABORTED: insufficient fuel
|       |-- Yes --> (layer 3)
|
|               Risk?
|               |-- LOW --> ☐ Launch to MOON
|               |-- MEDIUM --> ☐ Launch to MARS
|               |-- HIGH --> ☐ Launch to JUPITER
|
v
Play again? (yes/no)
|-- yes --> loop back to top
|-- no --> break --> END

```

Key Things to Remember

- **Slicing works on the name string:** `name[:3]` takes characters at index 0, 1, 2. `.upper()` makes them capitals. "Hamza" → "HAM"
- **Set ready = True before the for loop**, then set it to False inside the loop if the player types no for any item
- **The nested if/else has 3 levels of indentation.** Draw the decision tree on paper first before coding it — one wrong indent ruins all the outcomes
- **need and risk are set by the destination choice** — they carry forward into the launch verdict at the end

How to Build the Full Program Yourself

Follow these steps in order. Do not try to write everything at once. Build one piece, test it, then move to the next.

Step 1 — Write the Header

Start with the two institute header lines using `print()` and ANSI color codes. These appear once at the very top when the program runs.

```
text1 = "Institute: Benazir Bhutto Shaheed Human Resource Research &
Development Board"
text2 = "Course: Python Programming with AI"

print("\033[1;37;44m" + text1.center(100) + "\033[0m")
print("\033[1;30;47m" + text2.center(100) + "\033[0m")
```

Step 2 — Write the Rollercoaster Function

Type `def rollercoaster():` and indent everything inside it. Build the outer `while True` loop first. Then add the height validation `while` loop. Then add the age `if/elif/else`. Then the photo `while` loop. Then the billing. Test it by calling `rollercoaster()` at the bottom of your file.

Outer loop	<code>while True</code> — one full cycle per customer, breaks when <code>process another = no</code>
Counter vars	<code>count, revenue, seniors, photos</code> — set to 0 before the loop
Height check	<code>if h <= 120</code> → too short. <code>else</code> → proceed to age
Age groups	<code>elif 45 <= a <= 55</code> must come BEFORE <code>else</code> — senior before adult
Photo	<code>photo = 3</code> if <code>p == 'yes'</code> else 0
Summary loop	<code>for label, value in data:</code> — <code>data</code> is a list of tuples

Step 3 — Write the Treasure Island Function

Type `def treasure():` and wrap the entire game in `while True`. The game is purely nested `if/else` — no loops inside except the outer play again one. Focus on getting the indentation exactly right for each level.

Level 1	left or right? — if <code>choice == 'left'</code>
Level 2	swim or wait? — nested inside the left block
Level 3	red/blue/yellow door — nested inside the wait block

Play again	at the bottom of the while True loop — break if no
-------------------	--

Step 4 — Write the Space Mission Function

Type `def space():` and wrap in `while True`. Build each stage one at a time and test between stages. The trickiest part is the 3-layer nested if/else at the end — draw it on paper first.

Stage 1 — Name	while loop + slicing: <code>sign = name[:3].upper()</code>
Stage 2 — Health	while loop + if/else gate: <code>h >= 75</code> to continue
Stage 3 — Destination	while loop + if/elif/else: sets risk and need
Stage 4 — Equipment	for item in items + while loop validation + ready flag
Stage 5 — Fuel	while loop validation
Stage 6 — Verdict	3-layer nested if/else: ready → fuel → risk

Step 5 — Write the Main Menu Loop

After all three functions are written above it, write the main `while True` menu loop at the bottom. This loop calls the game functions and asks to return to menu after each game finishes.

Menu loop	while True — keeps showing until player exits
Input check	while True inner loop — validates 1/2/3/4 only
Call game	if/elif calls the right def function
After game	ask Back to menu? yes loops back, no breaks and exits

Step 6 — Test Everything

Run the program and try every possible path at least once. Use this checklist:

- Rollercoaster: height under 120 (rejected), child, teenager, senior, adult, with and without photos
- Rollercoaster: process 2+ customers, check the session summary totals are correct
- Treasure Island: right path (hole), left→swim (trout), left→wait→red, →blue, →yellow (win), →other
- Space Mission: health below 75 (rejected), moon with all equipment and enough fuel (launch), mars with missing equipment (aborted), jupiter with low fuel (aborted)
- Main menu: try typing an invalid choice like 5 or abc — the validation loop should catch it

- After each game: test yes (back to menu) and no (goodbye)

Quick Reference — Concepts Explained Simply

def function

A def gives a block of code a name so you can call it later. Write the function first, then call it by name. Without def, all three games would be mixed together in one giant file with no way to separate them.

```
def rollercoaster():  
    # all game code goes here, indented  
  
rollercoaster() # this line actually runs the function
```

while True + break

while True runs forever until you hit break. Use it when you don't know in advance how many times something should repeat — like asking for valid input, or looping a game until the player says stop.

```
while True:  
    h = input("Enter height: ")  
    if h.isdigit():  
        h = int(h)  
        break # valid input received — exit the loop  
    print("Numbers only!")
```

for loop over a list

A for loop runs once for every item in a list. The variable before in takes the value of each item one at a time. Great for equipment checklists or printing a summary.

```
items = ["Oxygen Tank", "Space Suit", "Navigation Computer"]  
  
for item in items:  
    print(f"Is your {item} ready?")  
    # runs 3 times — once for each item in the list
```

Slicing

Slicing cuts a piece out of a string using square brackets. name[:3] means from the start up to (not including) index 3 — so the first 3 characters. .upper() converts them to capitals.

```
name = "Hamza Khan"

sign = name[:3].upper()
# name[0] = "H"
# name[1] = "a"
# name[2] = "m"
# sign = "HAM"
```

Nested if/else

Nested means one if/else block sitting inside another. Each level adds one more condition that must be true before you reach it. The Space Mission launch verdict has 3 levels: equipment → fuel → risk.

```
if ready:                # layer 1
    if f >= need:         # layer 2
        if risk == "LOW": # layer 3
            print("Launch to MOON")
        elif risk == "MEDIUM":
            print("Launch to MARS")
        else:
            print("Launch to JUPITER")
    else:
        print("Aborted – not enough fuel")
else:
    print("Aborted – equipment missing")
```